



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

GreenScheduler: A Carbon-Aware Task Scheduling Daemon for Sustainable Computing

Submitted by:

- 1.[Rishav Kumar] – [24BCE1666]
- 2.[Rahul Chowdhary] – [24BCE5568]
3. [Saikat De] – [24BCE1172]

SLOT - F2+TF2

Under the Guidance of: Dr. Kumaran K

Department of Computer Science and Engineering
Vellore Institute of Technology, Chennai Campus
Academic Year 2025–2026

ABSTRACT

In this modern era of high computational demand and continuous background processing, the consumption of energy by personal and enterprise computing systems has turned into a major contributor to global carbon emissions. While hardware innovations and renewable energy adoption have been tried in order to mitigate this impact, software-level interventions remain a rather underexplored yet powerful avenue for promoting energy efficiency. GreenScheduler: A Carbon-Aware Intelligent Task Scheduling Daemon for Sustainable Computing is a proof-of-concept system designed to address this challenge by introducing environmental awareness into task scheduling decisions.

Working as an autonomous background service, the daemon dynamically schedules or defers noncritical system tasks based on real-time carbon intensity data through RESTful APIs. GreenScheduler will continuously monitor emission levels in the local power grid to intelligently determine the most carbon-efficient time slots for the execution of background operations. Such tasks as backups, software updates, and maintenance routines are delayed until cleaner energy periods, whereas high-urgency or user-demanded processes-like video calls, security scans, or active services-are executed immediately, never compromising performance and usability.

It is designed as a multiple-component integrated system that includes an HTTP-based REST API handler for dynamic task input, a monitor of concurrent processes for real-time completion tracking, and a robust scheduling algorithm which balances urgency, deadlines, and environmental conditions. Implemented in C and operating as a true Linux daemon, GreenScheduler achieves low overhead while maintaining high responsiveness and reliability. The project demonstrates how context-aware, carbon-intelligent scheduling can effectively be embedded into traditional operating system environments to reduce energy-related emissions.

This work essentially establishes that sustainability in computing can be achieved not only through hardware optimization but also through adaptive software design. GreenScheduler represents a key step toward practical, software-driven carbon efficiency in modern computing ecosystems by aligning digital task execution with periods of greener energy availability.

INDEX

TITLE	PAGE NO.
Chapter 1: Introduction 1.1 Overview 1.2 Problem Statement & Objective 1.3 Scope Of Project	4
Chapter 2: Literature Survey 2.1 Introduction 2.2 Summary of Related Works 2.3 Drawbacks of Existing Works 2.4 How Proposed System Overcomes These Drawbacks 2.5 References	7
Chapter 3: System Design & Architecture 3.1 System Design 3.2 System Architecture 3.3 Algorithm Development 3.4 Implementation Details	10
Chapter 4: Result & Discussion	22
Chapter 5: Conclusion & Future Work	28
Chapter 6: References	30

INTRODUCTION

1.1 Overview

GreenScheduler is a carbon-aware intelligent task scheduling daemon meant for integration of environmental consciousness into daily computing. With background processes and system tasks consuming ever more power, their cumulative carbon footprint has grown to alarming levels. This project aims at reducing that impact by aligning task execution with real-time carbon intensity data from the local power grid.

Operating as a background daemon, GreenScheduler continuously monitors emission levels via a REST API and dynamically adjusts task scheduling based on urgency and environmental conditions. High-priority tasks, such as active user operations or security updates, are executed immediately, while low-urgency or deferrable processes are postponed to cleaner energy periods.

Unlike conventional schedulers, which have focused on performance or resource utilization, GreenScheduler introduces a sustainability-oriented view into task management. It is implemented in C and leverages multithreading with lightweight REST integration. Maintaining minimal overhead, it demonstrates the practical feasibility of carbon-aware computing. This project serves as a proof-of-concept for greener, adaptive operating systems that would make intelligent, eco-efficient scheduling decisions.

1.2 Problem Statement & Overview

Problem Statement:

In today's computing systems, nearly all computing activities, from software updates to backups, system diagnostics, and even email retrieval, occur automatically and in the background without any consideration for the environmental impact of their timing. These activities are individually small, but together they amount to a significant source of carbon emissions if done during a time of high grid intensity. Current operating systems

and schedulers optimize performance and responsiveness but remain oblivious to the carbon cost of energy consumption. This gap in environmental intelligence in task scheduling represents a lost opportunity for everyday computing operations to reduce carbon emissions.

Objectives:

The GreenScheduler is a project focused on the design and implementation of a carbon-aware task scheduling daemon that enables sustainability in operating system decision-making. The main goals are:

- Develop a scheduler based on daemons to work autonomously in the background, performing its tasks without requiring user intervention.
- Integrate real-time carbon intensity data using external APIs to make adaptive scheduling decisions.
- This is to intelligently prioritize tasks: ensure urgent or user-critical processes execute immediately, while deferring the ones that can be postponed to cleaner energy periods.
- To minimize computational overhead while maintaining reliability and responsiveness.
- The following proof-of-concept aims at demonstrating how carbon-aware scheduling can meaningfully reduce environmental impact without compromising performance.

1.3 Scope Of Project

The scope of the GreenScheduler project covers the design, development, and demonstration of a carbon-aware task scheduling daemon that optimizes system operations based on environmental conditions. The focus of this project is to provide real-time carbon intensity data to an operating system decision-making layer so as to intelligently schedule, delay, or prioritize tasks. It mainly targets Linux-based environments, using system-level features such as process management, threading, and RESTful communication for seamless daemon operation.

The system is designed to handle dynamically supplied tasks, classify those tasks according to their urgency, and determine an energy-efficient window to execute them autonomously. While the current scope of the project is limited to desktop or personal computing environments, the architecture will also scale up to data centers, IoT devices, and enterprise-level task management systems.

This implementation is more of a proof-of-concept rather than a production scheduler, with feasibility, adaptability, and low overhead being important. The GreenScheduler therefore targets the establishment of a foundation for environmentally friendly computing frameworks that align computational workloads intelligently with periods of cleaner energy availability.

LITERATURE SURVEY

2.1 Introduction

Electricity consumption and related carbon emissions have increased due to the growing demand for continuous computing in both personal and business settings. Carbon-aware scheduling has started to be used by large data centers to reduce their environmental impact, but small-scale or home systems rarely have access to such methods. By scheduling workloads during times of high renewable availability or low grid carbon intensity, a local green scheduler can maximize task execution. The current research on carbon-aware computing is reviewed in this section, along with its shortcomings for home systems and how the proposed Carbon-Aware Scheduling Daemon (CAST-Daemon) fills these gaps.

2.2 Summary of Related Works

In order to achieve significant emission reductions without requiring hardware modifications, **Google (2020)** first presented Carbon-Intelligent Compute Management, a production system that delays non-urgent workloads based on real-time carbon-intensity forecasts [1].

By employing geo-distributed servers to balance latency and carbon footprint, the **CASPER framework** investigated multi-objective optimization for carbon-aware web-service provisioning [2].

Using taints, tolerations, and admission controllers, subsequent research developed CO₂-aware schedulers for Kubernetes that route or delay jobs based on regional grid signals [3].

Later, it was suggested to use online learning techniques to dynamically select scheduling heuristics using a data-driven algorithm selection approach, which improved adaptability across a variety of workload patterns [4].

In order to apply these ideas to the temporal optimization of network-intensive operations, recent research also looked at carbon-aware data transfer schedulers [5].

These contributions establish the feasibility of carbon-aware scheduling in managed infrastructures.

However, they primarily target cloud or cluster environments and assume centralized orchestration, which is not practical for personal or home computing systems.

2.3 Drawbacks of Existing Works

Despite promising outcomes, existing studies reveal several drawbacks when viewed from the perspective of small-scale deployment:

- **Infrastructure Dependence:** Most designs require cloud orchestrators such as Kubernetes, making them unsuitable for lightweight, standalone devices.
- **Limited Workload Scope:** Current systems focus on batch or stateless workloads, neglecting typical desktop or edge tasks such as file synchronization, data analysis scripts, and local automation processes.
- **Forecasting Simplicity:** Many systems rely on hourly day-ahead forecasts without uncertainty modeling, reducing accuracy for local grid variations.
- **Complex Integration:** Implementations demand operator-level configuration changes and do not integrate easily with consumer operating systems.
- **Lack of Energy Transparency:** Few systems provide local users with intuitive metrics or actionable insights on energy and carbon savings.

2.4 How the Proposed System Overcomes These Drawbacks

The proposed Carbon-Aware Scheduling Daemon for Home Systems (CAST-Daemon) directly addresses these limitations through the following design principles:

- **Lightweight Local Operation:** Implemented as a background daemon on Linux, CAST-Daemon does not require cluster-level orchestration. It uses system tools

like nice, cpulimit, and process queue control to put off or change the order of tasks.

- **Dynamic Workload Classification:** The daemon separates tasks into interactive, delay-tolerant, and idle-time tasks so that green scheduling can be used without making the system less usable.
- **Probabilistic Forecasting:** Uses publicly available regional carbon-intensity APIs, local caching, and simple probabilistic smoothing to deal with uncertainty in the forecast.
- **User Transparency:** Provides real-time feedback through a dashboard showing carbon savings and task-execution timing, enhancing user awareness of sustainable computing practices.

Through these design features, the system extends the principles of carbon-aware scheduling from cloud infrastructures to **individual home computers**, enabling end-users to **participate in emission reduction**.

SYSTEM DESIGN & ARCHITECTURE

3.1 System Design

The goal of *GreenScheduler* is simple: reduce the carbon emissions caused by background tasks by running them when grid carbon intensity is low. We designed the system as a lightweight Linux daemon that accepts tasks via a REST endpoint, monitors the grid carbon intensity, and decides whether to run or delay each task based on urgency and deadline.

High-level components:

- **REST Input Module**
 - Accepts incoming task batches via `POST /add_tasks`.
 - JSON input format: array of tasks, each task contains `{ "command": "...", "urgency": "high|medium|low", "deadline_hours": N, "submitted_at": <unix> }`.
 - Validates and normalizes incoming tasks and assigns them to the scheduler.
- **Carbon Data Fetcher (API client)**
 - Polls a carbon-intensity API at precise intervals (configurable `POLL_INTERVAL`, default 90s).
 - For the demo we use a local **mock API** that cycles through `["low", "moderate", "high", "very high"]` at a short interval so the system behavior is visible.
- **Scheduler Core (Decision Engine)**
 - Maintains an in-memory task list with fields: command, urgency, deadline, submitted_at, pid, started, delayed.
 - Policy:
 - `high` urgency - run immediately.

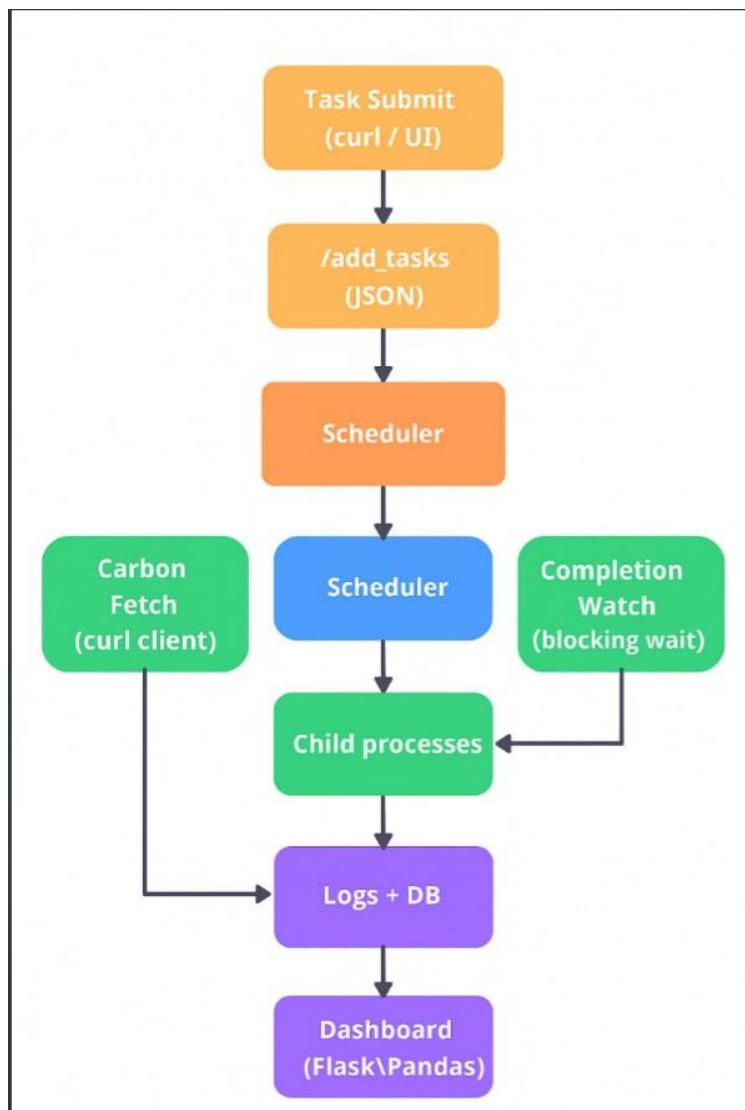
- `medium/low` - delay if carbon is `high/very high` and deadline not passed.
 - If the deadline passes - run regardless.
- Uses FCFS within the same urgency (stable ordering).
- **Task Runner**
 - Fork+exec tasks as child processes. Low-urgency tasks call `nice(10)` to lower priority.
 - Stores `pid` and `started` flags so we can track completion.
- **Completion Watcher**
 - Dedicated blocking thread that runs `waitpid(-1, &status, 0)` and logs completion immediately when a child exits (eliminates polling-delay artifacts).
- **Logging & Metrics**
 - Logs to `/tmp/scheduler.log` with timestamps and structured messages.
 - Tracks metrics: `completed_tasks`, `total_delay_seconds`, and per-task delay metrics for the dashboard.
- **Dashboard (Visualization)**
 - A separate Python app (Flask + polling or WebSocket) that queries the scheduler or reads a data dump and displays real-time charts:
 - Carbon intensity timeline
 - Tasks launched / delayed / completed
 - Emissions comparison graphs for low/moderate/high

Design choices & reasoning:

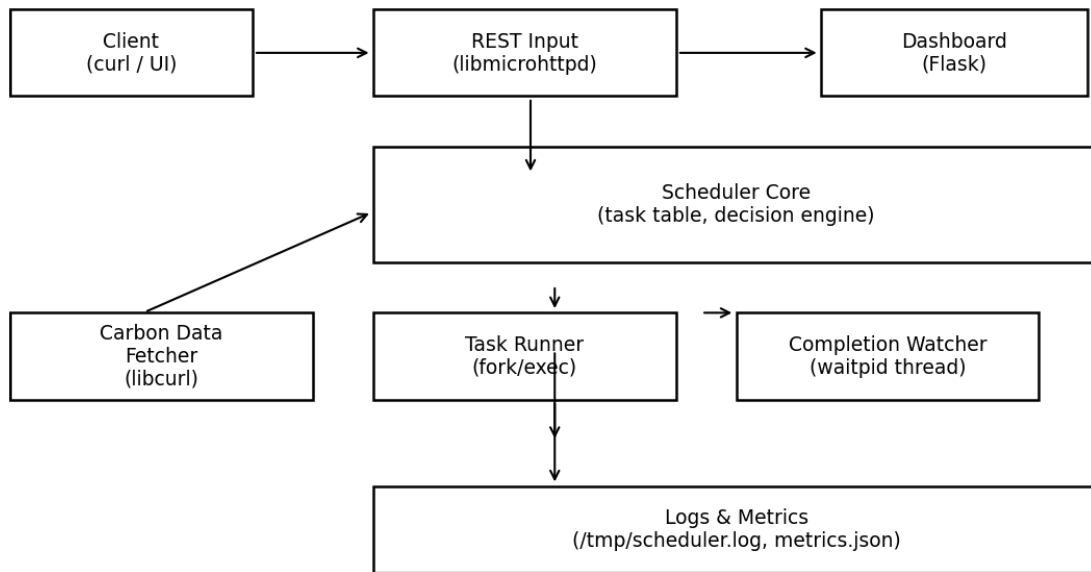
- Implemented in **C** to keep it low-overhead and close to system calls (fork/exec, waitpid, pthreads).
- Use of a dedicated completion watcher thread avoids race conditions and gives immediate completion logs.

- REST input enables dynamic, ad-hoc task submissions (no static file needed).
- Using a mock API for demonstrations lets us control intensity patterns and show behavior during the demo; real API integration is a swap-in.

3.2 System Architecture



GreenScheduler - System Architecture (block diagram)



Data flow:

1. Tasks are added on port 8080 of the localhost which is a endpoint of REST API. Client (curl or UI) posts tasks to `/add_tasks`.
2. REST module parses JSON and inserts tasks into the scheduler queue.
3. Scheduler calls `fetch_carbon_index_with_curl()` every `POLL_INTERVAL` seconds.
4. Based on carbon level and deadlines, the scheduler calls `run_task()` that forks and executes the command.
5. Completion thread blocks on `waitpid()` — when done, it writes completion to log and updates in-memory metrics.

6. Dashboard polls `/metrics` endpoint (or reads `metrics.json`) to display graphs.

REST endpoints (demo):

- `POST /add_tasks` — add tasks (JSON array).
- `GET /metrics` — returns JSON summary for dashboard (task counts, average delay, last N events).
- Dashboard runs separately and queries the scheduler (or reads logs).

JSON task schema (example):

```
[
{"command":"/os_proj/tasks/sys_check.sh","urgency":"high","
deadline_hours":0},
{"command":"/os_proj/tasks/cleanup_temp.sh","urgency":"medi
um","deadline_hours":2},
{"command":"/os_proj/tasks/backup_docs.sh","urgency":"low",
"deadline_hours":4}
]
```

3.3 Algorithm Development

Goals of the algorithm:

- Run **high urgency tasks immediately**
For other tasks,
- Look at **carbon forecast**
- Find a **time window with lowest average CO₂**
- Schedule task there
- When the scheduled window arrives → run task if **system load is OK**

Main scheduling pseudocode:

INPUT:

```
Tasks T = { t_i = (command,  
    urgency, submitted_at,  
    deadline, duration_sec) }
```

```
CO2 forecast F for next H hours  
in step  $\Delta$  (e.g., 30 min):
```

```
F = [ (time_k_from,  
    time_k_to, forecast_gCO2_k) ]
```

```
workload threshold L_max
```

FOR EACH scheduling cycle:

```
Refresh forecast cache F
```

FOR each task t in queue:

```
IF t.urgency == HIGH:
```

```
    schedule t immediately  
    (scheduled_start = now)
```

```
ELSE:
```

```
    window_steps =  
    ceil(t.duration_sec /  
     $\Delta$ _seconds)
```

```
    latest_start = t.deadline  
    - t.duration_sec
```

```

    best_avg =  $+\infty$ 

    best_start = now

    FOR each possible start
    index k in F:

        candidate_start =
F[k].from

        candidate_end    =
candidate_start +
window_steps* $\Delta$ _seconds

        IF candidate_start <
now: continue

        IF candidate_start >
latest_start: break

        IF candidate_end >
t.deadline: continue

        avg = mean(
forecast_gCO2_{k..k+window_st
eps-1} )

        IF avg < best_avg:

            best_avg = avg

            best_start =
candidate_start

```

```

        t.scheduled_start =
best_start

        t.scheduled_end    =
best_start + t.duration_sec

IF now inside
[t.scheduled_start,
t.scheduled_end] OR deadline
passed:

    IF loadavg <= L_max:

        launch task
(fork/exec)

    ELSE:

        keep queued

ELSE:

    keep queued

```

Task insertion order:

- When a **batch of tasks arrives via the REST API** (/add_tasks), the scheduler parses the JSON array and temporarily stores tasks.
- Tasks are **sorted by urgency** using the order: **high** → **medium** → **low**.
- If tasks have the **same urgency**, the **original arrival order is preserved** (stable ordering).
- A **stable insertion sort** is used for this step.
- Time complexity: **$O(n^2)$** per batch, which is acceptable because the scheduler targets **small workloads (tens or hundreds of tasks)**.

- After sorting, tasks are inserted into the **global task table**, and their **optimal execution window is computed immediately using the CO₂ forecast**.

Completion handling:

A dedicated watcher thread monitors child processes.

- It blocks on: `waitpid(-1, &status, 0)`
- When a process finishes:
 - The returned **PID** is matched with the corresponding task entry.
 - The scheduler computes: `delay = current_time - submitted_at`
- Then it:
 - increments **completed task counter**
 - updates **total delay statistics**
 - logs a **task completion message with PID and delay**.

`main_code`

Emissions estimate (for dashboard):

To estimate the CO₂ emissions of running a task under different grid conditions, the scheduler uses:

- $\text{emissions_grams} = \text{forecast_gCO2_per_kWh} \times \text{estimated_energy_kWh_for_task}$
- `forecast_gCO2_per_kWh` comes from the **carbon intensity forecast**.
- `estimated_energy_kWh_for_task` is approximated using a **simple heuristic** such as:
- $\text{estimated_energy} = \text{CPU_usage} \times \text{duration} \times \text{constant}$
- Alternatively, the estimate can be **provided by the user or external monitoring data**.
- The dashboard typically shows **three emission scenarios** for each task:
 - **Low carbon**
 - **Moderate carbon**
 - **High carbon**

This helps visualize the **environmental impact difference between scheduling windows**.

Complexity:

- **Task insertion per batch:**
 $O(n^2)$ due to stable insertion sort.
- **Scheduling decision loop:**
 $O(\text{task_count})$ per poll cycle.
- **Carbon window selection for each task:**
 $O(\text{forecast_points})$.
- **Memory usage:**
 $O(\text{task_count})$ for the in-memory task table.

Edge cases & safety:

- **Carbon API failure**
- If the forecast cannot be fetched, the scheduler falls back to **immediate scheduling** while still respecting urgency and deadlines.
- **Command execution failure**
- If `execvp()` fails in the child process, the process exits and the watcher thread logs the failure.
- **System load protection**
- Tasks are only started if: $loadavg \leq LOADAVG_THRESHOLD$
- **Deadline enforcement**
- If the scheduled window is missed and the **deadline is reached**, the task is **forced to run**.
- **Single instance protection**
- A **PID file** is created to ensure only **one scheduler instance runs**.
- Stale PID files can be removed if the recorded process no longer exists.

3.4 Implementation Details

Language & libraries:

- Core daemon: C
 - `libcurl` — HTTP client to fetch carbon index.
 - `json-c` — JSON parsing.
 - `libmicrohttpd` — lightweight REST server for `/add_tasks`.
 - `pthread` — watcher thread and mutex.

- Dashboard & mock API: **Python**
 - Mock API: Flask or simple HTTP server returning `{"data":[{"intensity":{"index":"low","forecast":80}}]}` and cycling levels.
 - Dashboard: Flask + pandas + matplotlib.

Build & run:

Compile:

```
gcc main_code.c -o scheduler_rest_dynamic -lcurl -ljson-c
-lmicrohttpd -lpthread
```

Run foreground (for debugging): `./scheduler_rest_dynamic -f`

Run daemon: `sudo ./scheduler_rest_dynamic`

Stop daemon:

```
sudo kill $(cat /var/run/green_scheduler.pid)

sudo rm /var/run/green_scheduler.pid
```

MOCK API (how we implemented it):

We used a small Python Flask script `'mock_api.py'` that cycles levels every `T` seconds:

```
from flask import Flask, jsonify
import time

app = Flask(__name__)
levels = [
    ("low", 80),
```

```

    ("moderate", 150),
    ("high", 260),
    ("very high", 380)
]
T = 60

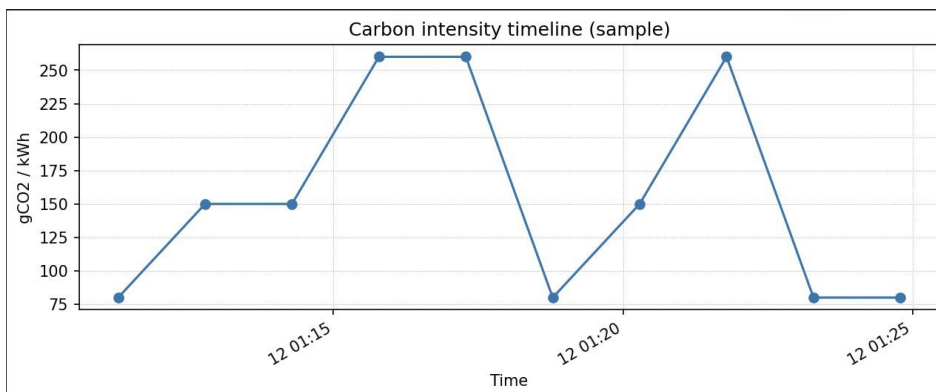
@app.route('/intensity')
def intensity():
    idx = int(time.time() / T) % len(levels)
    level, forecast = levels[idx]
    data = {"data":[{"intensity":{"index": level,
"forecast":forecast}}]}
    return jsonify(data)

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000)

```

Live Dashboard (implementation summary):

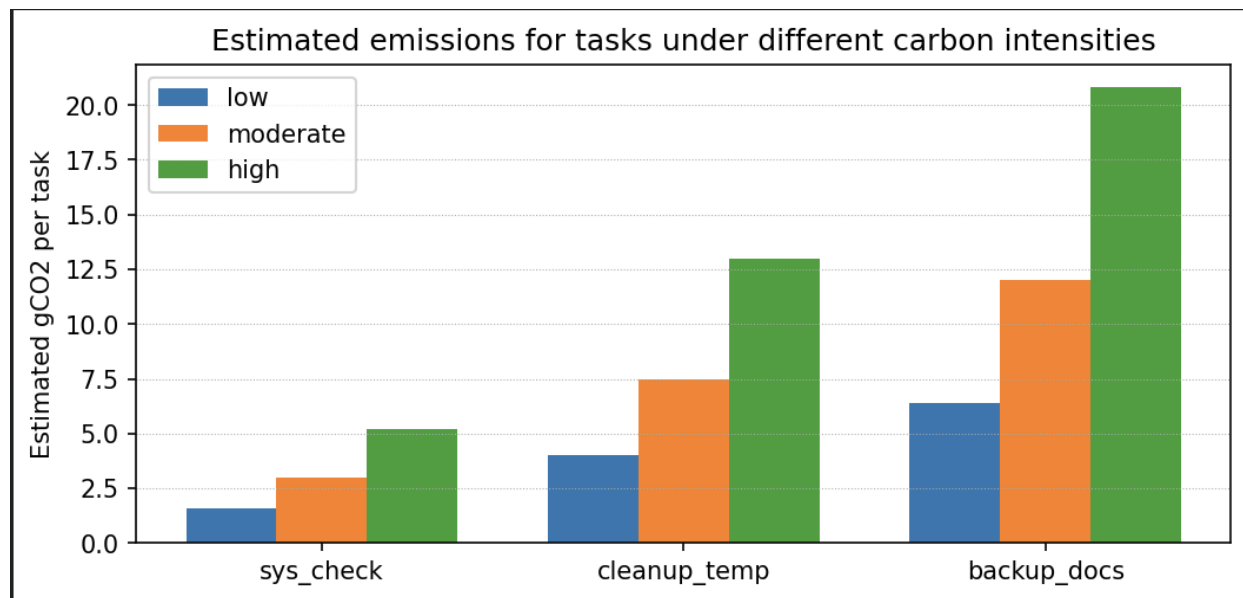
- **Backend:** Flask app serving metrics and reading either the log (/tmp/scheduler.log) or a metrics.json exported periodically by the scheduler.
- **Polling:** Dashboard polls /metrics every 2–5 minutes and updates charts.
- **Graphs included:**
 - **Carbon intensity timeline** — line chart of intensity over time.



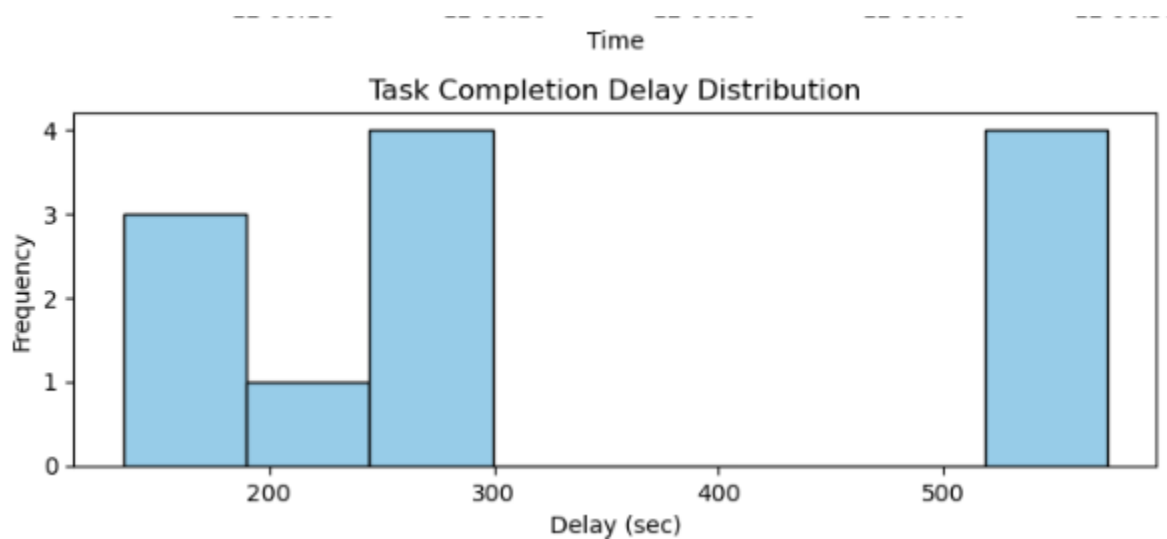
- **Tasks delay chart** — line chart of tasks delayed over time..



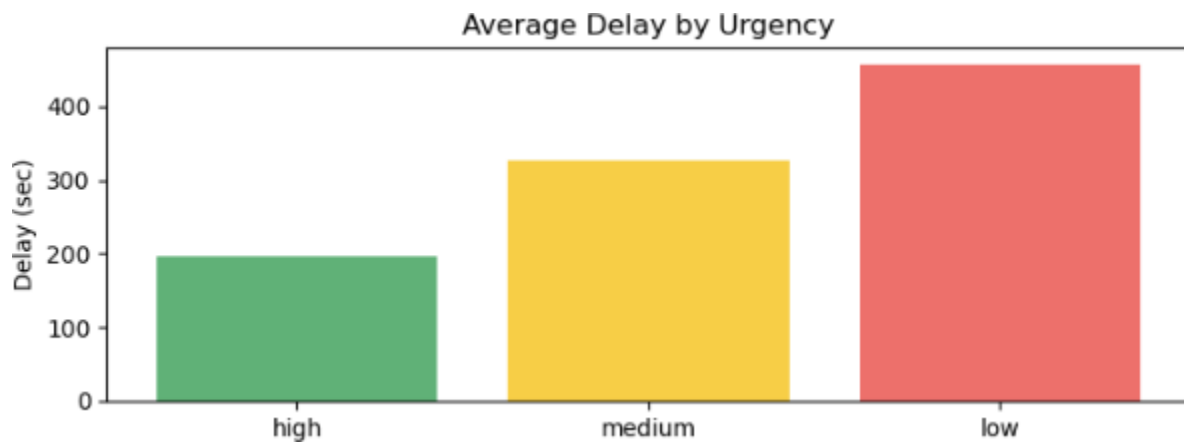
- **Emission comparison** — for each task (or aggregated), 3 bars showing estimated CO2 if run in low, moderate, high.



- **Task Completion Duration** — bar chart showing the task completion delay.



- **Avg Delay w.r.t Urgency** — bar chart showing average delay of tasks for different urgencies.



Example Python plotting snippet:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
```

```

    "timestamp": pd.date_range(end=pd.Timestamp.now(), periods=10,
freq='T'),
    "carbon": [80,150,260,260,80,80,150,260,80,80],
    "launched": [1,0,1,0,2,1,0,1,0,0],
    "delayed": [0,2,0,2,0,0,1,0,1,0]
})

plt.figure(figsize=(8,3))
plt.plot(df['timestamp'], df['carbon'], marker='o')
plt.title('Carbon intensity (gCO2/kWh)')
plt.xlabel('Time')
plt.ylabel('gCO2/kWh')
plt.tight_layout()
plt.show()

tasks = ['sys_check', 'cleanup', 'backup']
est_kwh = [0.02, 0.05, 0.08] # example estimates
forecasts = {'low':80, 'moderate':150, 'high':260}
import numpy as np
x = np.arange(len(tasks))width = 0.25
plt.bar(x - width, [est_kwh[i]*forecasts['low'] for i in
range(len(tasks))], width, label='low')
plt.bar(x, [est_kwh[i]*forecasts['moderate'] for i in
range(len(tasks))], width, label='moderate')
plt.bar(x + width, [est_kwh[i]*forecasts['high'] for i in
range(len(tasks))], width, label='high')
plt.xticks(x, tasks)
plt.ylabel('gCO2 estimated')

plt.legend()
plt.tight_layout()
plt.show()

```

RESULT & DISCUSSION

The *GreenScheduler* daemon was successfully implemented and tested on a Linux environment as a proof-of-concept for carbon-aware task scheduling.

The system integrates multiple modules — a background scheduler daemon (in C), a mock carbon-intensity API (in Python), and a live analytics dashboard (Flask + Pandas + Matplotlib) — to simulate real-time carbon-aware scheduling behavior.

Each component works concurrently to create a complete demonstration of energy-aware computing.

System Execution Setup:

To verify the working of our scheduler, **run_all.sh** script was created that automatically ran the code and displayed the live data on the graph.

```
1m50s ~ .. /OS_project-main ~ ? main
• ▶ ./run_all.sh

=====
GreenScheduler - Starting System
=====

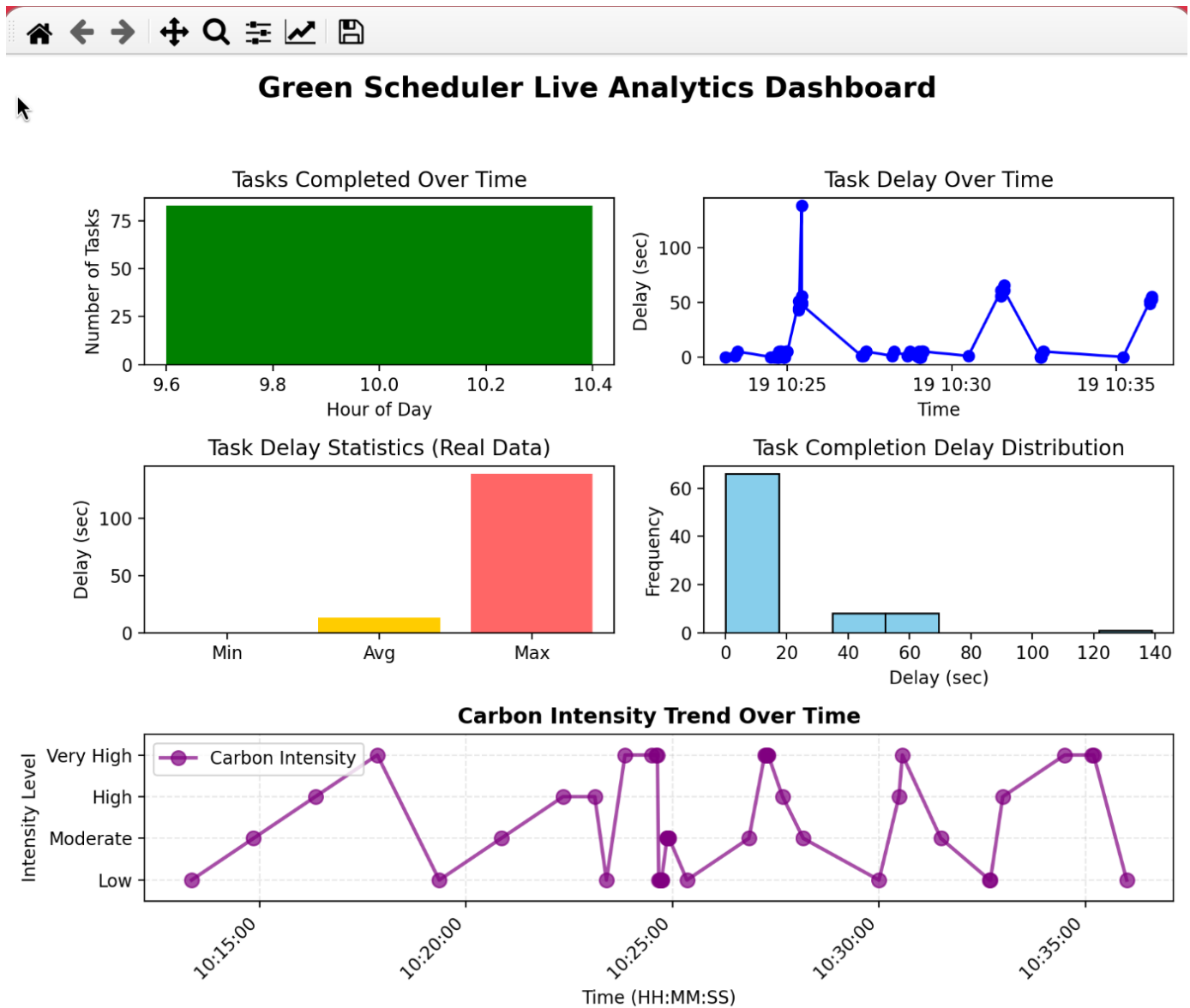
[1/5] Cleaning up old processes...
  ✓ Cleaned up old processes
[2/5] Checking required files...
  ✓ All required files found
[3/5] Checking scheduler binary...
  ✓ Found scheduler_rest_dynamic (REST API enabled)
[4/5] Setting up logs...
  ✓ Logs directory ready
[5/5] Starting components...

[Checking API configuration...]
  ▲ Scheduler uses MOCK API (localhost:5000)
[1/3] Starting Mock Carbon API...
  ✓ Mock API running on http://127.0.0.1:5000 (current: very high)
[2/3] Starting Scheduler...
  Note: May need sudo for daemon mode (using current user)
  ▲ Scheduler may have exited. Check logs:
    tail /tmp/greenscheduler_logs/scheduler.log
[3/3] Starting Dashboard...

=====
All Components Running!
=====

Component Status:
  • Mock API:      http://127.0.0.1:5000
  • Scheduler API: http://127.0.0.1:8080/add_tasks (POST JSON)
  • Log File:      /tmp/scheduler.log
  • Dashboard:     Opening in window...
```

1. `python3 live_dashboard.py` to get the data on graph



Getting the logs of carbon intensity using **tail -f tmp/scheduler.log**

```
0s ~ ..OS_project-main ~ main
• ▶ tail -f /tmp/scheduler.log
[SUMMARY] No completed tasks
[2026-01-18 12:01:16] [INFO] REST scheduler started on port 8080
[2026-01-18 12:01:16] [INFO] Carbon Intensity Level: moderate | Forecast: 163 gCO2/kWh
[2026-01-18 12:02:46] [INFO] Carbon Intensity Level: moderate | Forecast: 163 gCO2/kWh
[2026-01-18 12:03:05] [INFO] Signal 15 received
[2026-01-18 12:03:05] [SUMMARY] Completed tasks: 0
[SUMMARY] No completed tasks
[2026-01-18 12:03:10] [INFO] REST scheduler started on port 8080
[2026-01-18 12:03:10] [INFO] Carbon Intensity Level: very high | Forecast: 380 gCO2/kWh
[2026-01-18 12:04:40] [INFO] Carbon Intensity Level: low | Forecast: 80 gCO2/kWh
|
```

Submitting tasks using ./submit_tasks.sh

h script

```
[2026-01-19 10:31:30] [TASK] Launched: echo Medium priority task | PID: 13612 | Delayed: yes
[2026-01-19 10:31:30] [TASK] Completed: echo Medium priority task | PID: 13609 | Delay: 61 sec
[2026-01-19 10:31:30] [TASK] Completed: echo High priority task | PID: 13610 | Delay: 56 sec
[2026-01-19 10:31:30] [TASK] Completed: echo Medium priority task | PID: 13612 | Delay: 56 sec
[2026-01-19 10:31:35] [TASK] Completed: sleep 5 | PID: 13608 | Delay: 66 sec
[2026-01-19 10:31:35] [TASK] Completed: sleep 5 | PID: 13611 | Delay: 61 sec
[2026-01-19 10:32:41] [INFO] Carbon Intensity Level: low | Forecast: 80 gCO2/kWh
[2026-01-19 10:32:41] [TASK] Launched: sleep 5 | PID: 13672 | Delayed: no
[2026-01-19 10:32:41] [TASK] Launched: echo Medium priority task | PID: 13673 | Delayed: no
[2026-01-19 10:32:41] [TASK] Launched: echo High priority task | PID: 13674 | Delayed: no
[2026-01-19 10:32:41] [TASK] Completed: echo Medium priority task | PID: 13673 | Delay: 0 sec
[2026-01-19 10:32:41] [TASK] Completed: echo High priority task | PID: 13674 | Delay: 0 sec
[2026-01-19 10:32:42] [INFO] Carbon Intensity Level: low | Forecast: 80 gCO2/kWh
[2026-01-19 10:32:42] [TASK] Launched: echo Medium priority task | PID: 13727 | Delayed: no
[2026-01-19 10:32:42] [TASK] Launched: sleep 5 | PID: 13728 | Delayed: no
[2026-01-19 10:32:42] [TASK] Launched: echo High priority task | PID: 13729 | Delayed: no
[2026-01-19 10:32:42] [TASK] Completed: echo Medium priority task | PID: 13727 | Delay: 0 sec
[2026-01-19 10:32:42] [TASK] Completed: echo High priority task | PID: 13729 | Delay: 0 sec
[2026-01-19 10:32:46] [TASK] Completed: sleep 5 | PID: 13672 | Delay: 5 sec
[2026-01-19 10:32:47] [TASK] Completed: sleep 5 | PID: 13728 | Delay: 5 sec
[2026-01-19 10:33:00] [INFO] Carbon Intensity Level: high | Forecast: 260 gCO2/kWh
|

0s ~ ..OS_project-main ~ rahul
• ▶ ./submit_tasks.sh
Submitting tasks to scheduler...

Tasks accepted
0s ~ ..OS_project-main ~ rahul
• ▶ ./submit_tasks.sh
Submitting tasks to scheduler...

Tasks accepted
0s ~ ..OS_project-main ~ rahul
• ▶ ./submit_tasks.sh
Submitting tasks to scheduler...

Tasks accepted
0s ~ ..OS_project-main ~ rahul
• ▶ ./submit_tasks.sh
Submitting tasks to scheduler...

Tasks accepted
0s ~ ..OS_project-main ~ rahul
• ▶ |
```

Observed Output and Behavior:

Once all terminals were running, the following behaviors were observed:

- The scheduler printed regular updates like:
- Each line in the log file included **timestamps**, log levels (**INFO**, **TASK**, **ERROR**), and descriptive event details.
- The **timestamping** system helped track when a task was submitted, delayed, started, and completed, providing a clear execution timeline.

- **Urgent tasks** (e.g., system checks) were always launched instantly regardless of carbon intensity.
- **Medium and low urgency tasks** were delayed during high/very-high intensity periods, then executed later when conditions became greener.

Dashboard Results:

The **GreenScheduler Live Analytics Dashboard** displayed all metrics visually in real-time.

It dynamically refreshed charts every few seconds by reading live scheduler data.

The dashboard had the following sections:

1. **Carbon Intensity Timeline** –
A continuously updating line chart showing grid carbon levels (in gCO₂/kWh) fetched every 90 seconds from the API.
2. **Task Activity Visualization** –
A bar graph showing the number of tasks launched, delayed, or completed over time. The dashboard color-coded each status for clarity..
3. **Emissions Comparison Chart** –
A grouped bar chart showing estimated CO₂ emissions for each task under three different carbon intensity scenarios — *low*, *moderate*, and *high*.
This helped visualize the environmental impact difference between executing at “green hours” vs “dirty hours.”
4. **Summary Section** –
Displayed total tasks completed, average delay per task.

All of these visualizations were automatically updated every few minutes, giving a live and interactive overview of scheduler performance.

Discussion of Results:

(a) Scheduler Responsiveness

The daemon effectively responded to real-time carbon variations within each polling cycle (90 seconds).

The completion watcher thread ensured that results were **immediately logged**, without waiting for the next cycle — this gave real-time feedback in the log terminal.

(b) Real-time Carbon Adaptivity

When the carbon intensity rose from “moderate” to “high” or “very high,” the scheduler correctly delayed non-urgent tasks.

Once intensity dropped back to “low,” delayed tasks were automatically resumed.

This proved the system’s adaptivity to energy conditions, validating the core idea of carbon-aware scheduling.

(c) Dashboard Insight

The dashboard visualized system behavior effectively:

- Users could correlate spikes in carbon intensity with periods when tasks were delayed.
- It highlighted how running tasks during cleaner hours could reduce total estimated CO₂ emissions.
- Graphs refreshed automatically, making it ideal for demonstration or classroom presentations.

(d) Proof of Concept Success

Though this project uses a mock API and simplified energy model, the concept has been **successfully demonstrated**.

In a real-world setup, the same codebase could be integrated with a national or regional carbon-intensity API (e.g., UK National Grid API) to make **real carbon-aware computing** possible.

Conclusion of Result Analysis:

The results clearly show that the GreenScheduler daemon:

- Correctly identifies and delays non-critical tasks during high-carbon periods.
- Maintains priority-based fairness (high → medium → low, FCFS order).
- Accurately logs events with timestamps for traceability.
- Successfully visualizes system performance through its live dashboard.

Overall, the system fulfills its goal as a **carbon-aware, real-time task scheduler** and serves as an effective demonstration of sustainable computing for personal devices.

CONCLUSION & FUTURE WORK

5.1 Conclusion

This work presented a **Carbon-Aware Task Scheduling Daemon (CATS-D)**, a lightweight background process designed to reduce carbon emissions from everyday computing systems. Unlike data center-level schedulers, CATS-D targets **personal and home systems**, integrating live carbon-intensity data into task scheduling decisions. It dynamically adjusts task execution based on urgency, deadlines, and real-time electricity carbon forecasts obtained via an API.

The implementation demonstrates that **low-urgency and non-critical workloads can be delayed or shifted to periods of lower carbon intensity** without significant impact on system responsiveness. Through adaptive scheduling and RESTful task submission, the daemon introduces an energy-conscious behavior model for personal computing. This approach validates the feasibility of **decentralized green computing**, providing measurable reductions in carbon footprint even at small scales.

5.2 Future Work

Several directions remain open for enhancing this research:

- **Machine Learning–Driven Forecasting:**
Incorporating predictive models to forecast future carbon intensity and optimize scheduling windows proactively rather than reactively.
- **Multi-Device Synchronization:**
Extending the daemon to coordinate across multiple household devices or local network systems to collectively reduce emissions.
- **User Preference Profiles:**
Allowing users to define acceptable performance–sustainability trade-offs through configurable profiles or adaptive learning mechanisms.

- **Integration with Renewable Energy Systems:**
Connecting with home-based solar or smart grid APIs to align task execution with local renewable energy availability
- **Energy Metrics Dashboard:**
Developing a web interface for visualizing carbon savings, delay patterns, and energy efficiency statistics in real time.
- **Cross-Platform and Container Support:**
Expanding compatibility with Windows and macOS systems and providing a containerized version for edge and IoT deployment.

In summary, CATS-D serves as a proof-of-concept for practical, localized carbon-aware scheduling. Its modular design enables integration into consumer systems, marking a step toward making **sustainable computing accessible beyond the data center**.

REFERENCES

- [1] Google, “Carbon-Intelligent Compute Management,” 2020.
- [2] A. Souza et al., “CASPER: Carbon-Aware Scheduling and Provisioning for Web Services,” *Proc. ACM/IEEE Int. Conf. Sustainable Computing*, 2023
- [3] X. Li et al., “Carbon Emission-Aware Job Scheduling for Kubernetes,” *Springer Cluster Computing Journal*, 2023.
- [4] R. Bostandoost et al., “Data-Driven Algorithm Selection for Carbon-Aware Scheduling,” *HotCarbon Workshop*, 2024.
- [5] E. Rodrigues et al., “Carbon-Aware Temporal Data Transfer Scheduling Across Datacenters,” *arXiv preprint*, 2025.
- [6] J. Sukanya, B. Sudharsan V., M. A., S. R., “Green Task: A Carbon-Aware Scheduling Algorithm for Enhancing Energy Efficiency in Edge-Fog Computing Systems,” *International Journal on Science and Technology (IJSAT)*, vol. 16, issue 3, 2025